

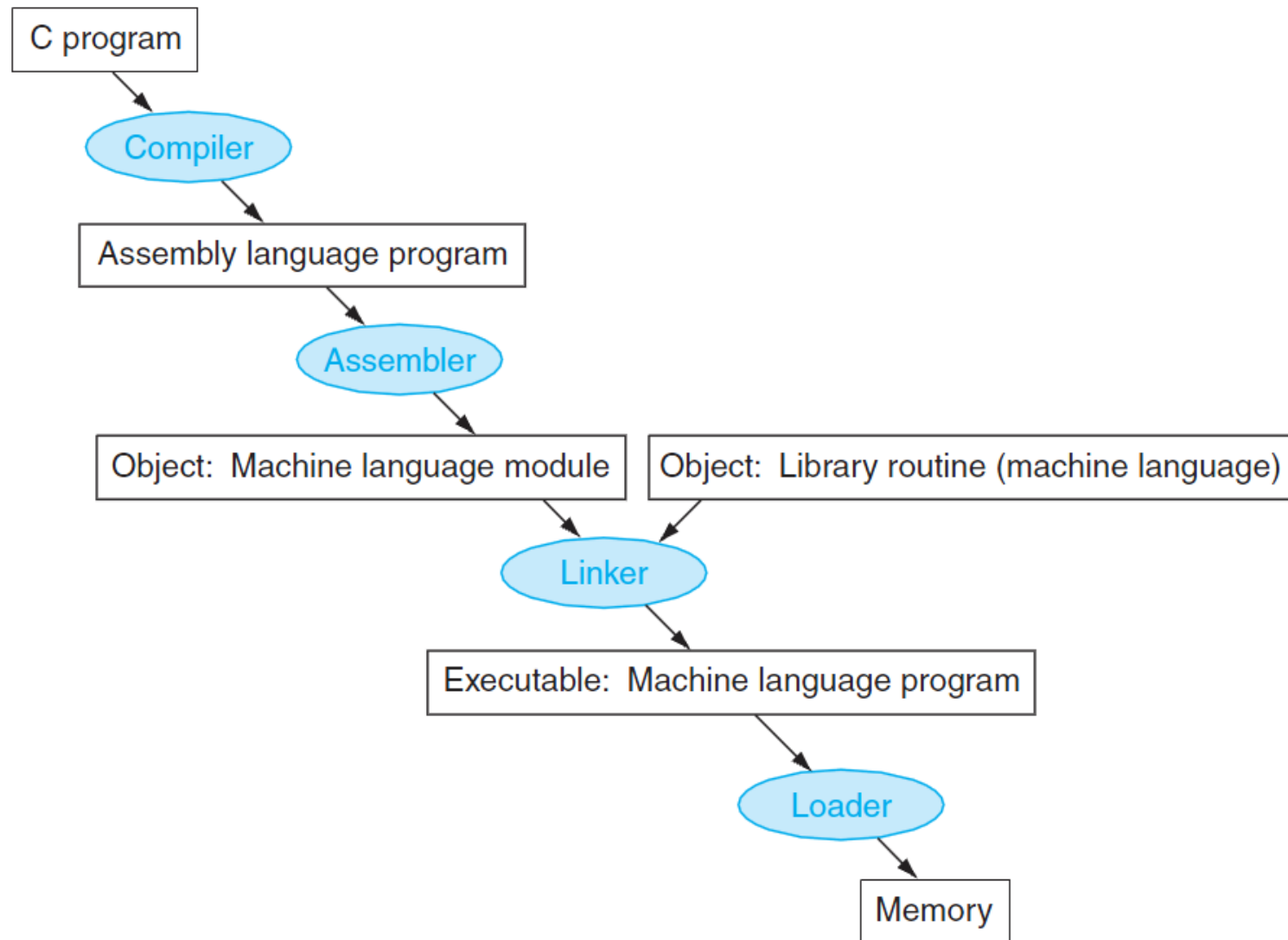
Advanced Computer Systems (CIS 570)

Computer Execution Cycle

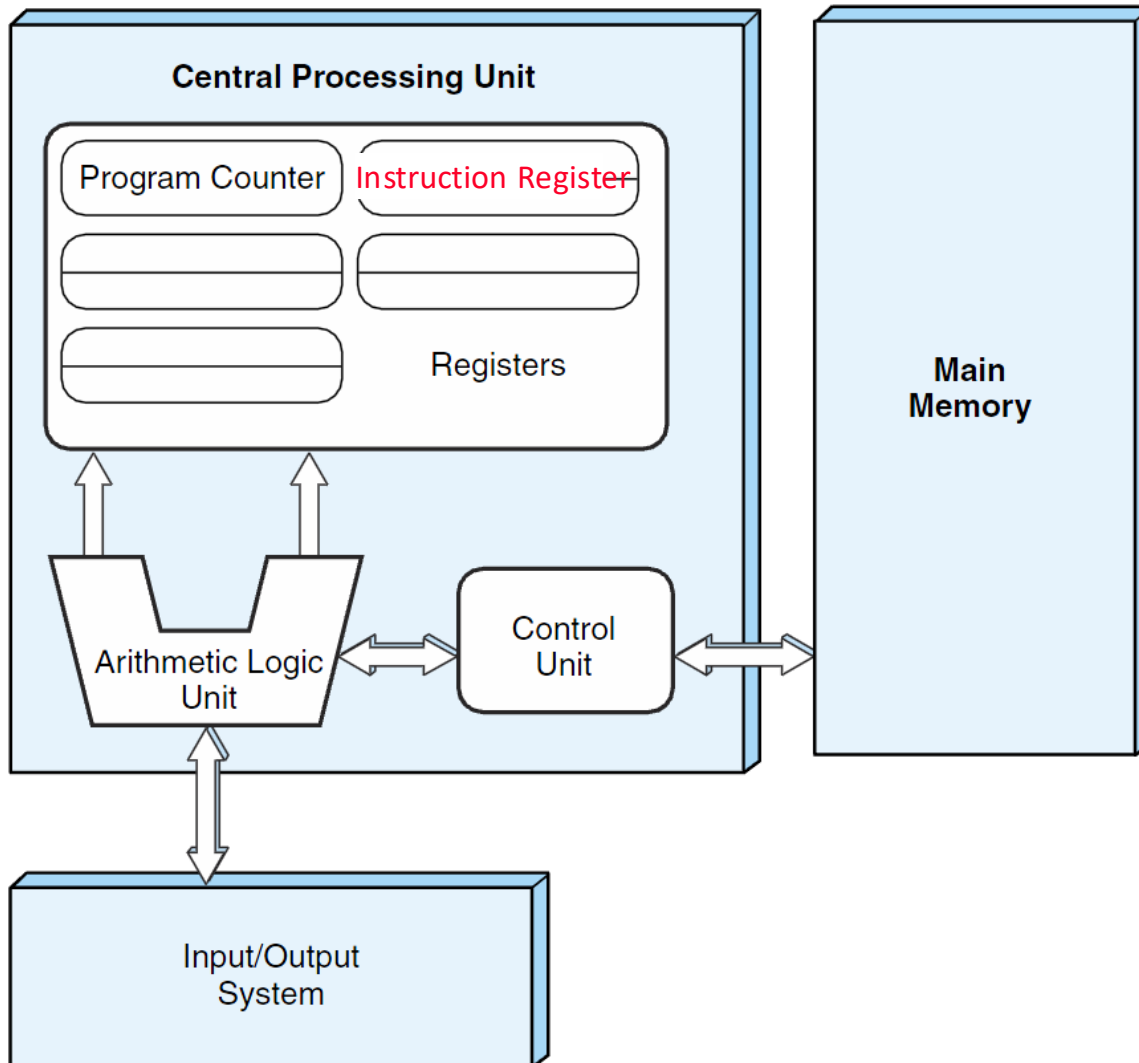
Slides are adopted from:

- Patterson and Hennessy, Computer Architecture a Quantitative Approach (Fifth edition)
- Notes by Dr. O. Mutlu, CMU
- Patterson and Hennessy, Computer Organization and Design: The Hardware/Software Interface, Fourth Edition
- Notes by Dr. M. Geiger, UMA.
- Vincent Heuring and Harry Jordan, Computer Systems design and Architecture, Second Edition

Executing HLL Programs



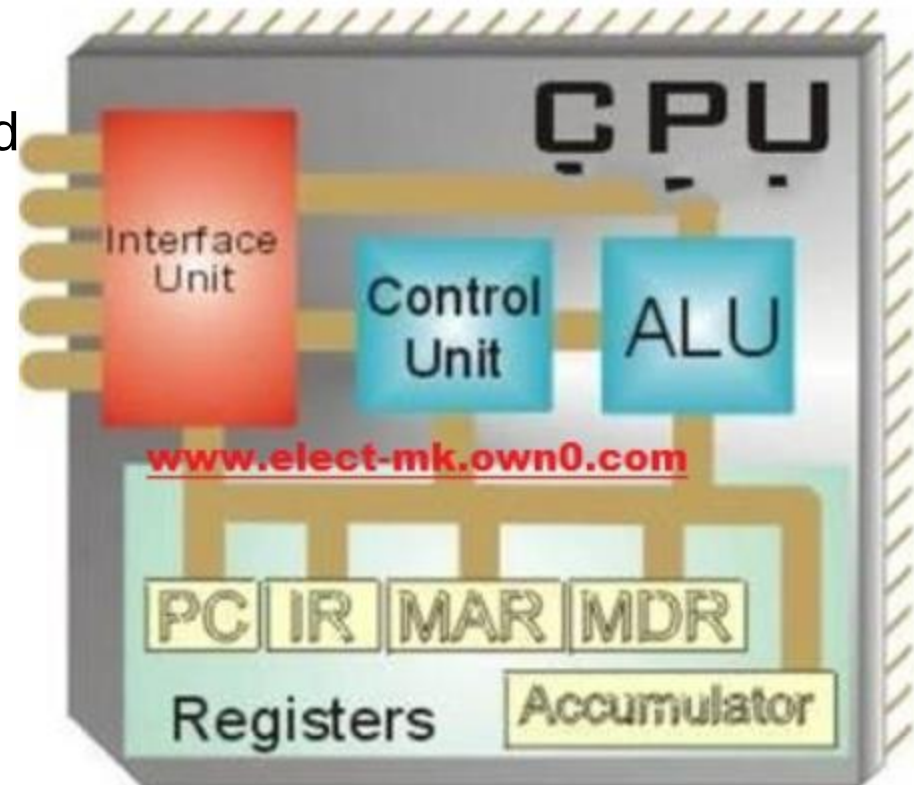
Computer System



- **Program Counter:** address of next instruction to be fetched
- **Instruction Register:** instruction currently being decoded and/or executed
- **CPU State:** a snapshot of the values in **all** different registers

Microprocessor

- **Arithmetic Logic Unit:** digital circuit that performs arithmetic and logical operations
- **Control Unit:** provides timing and control signals
- **Memory Address Register:** memory address of data to be read/written to RAM
- **Memory Data Register:** data to be written/read from RAM
- **Program Counter:** address of next instruction to be fetched
- **Instruction Register:** instruction currently being executed or decoded

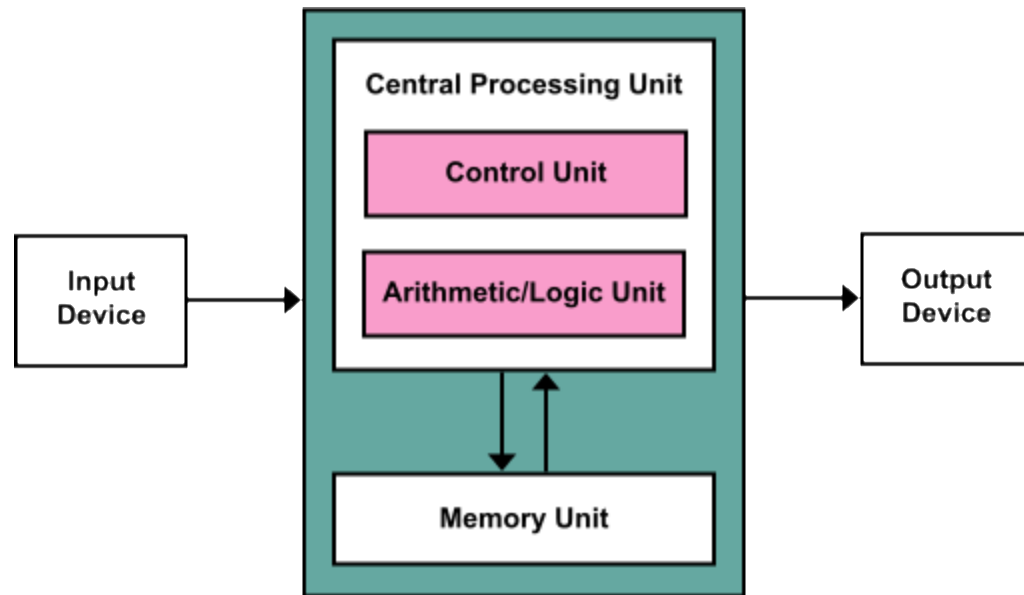


Von Neumann Model/Architecture

1945



John von Neumann



Von Neumann: Architecture

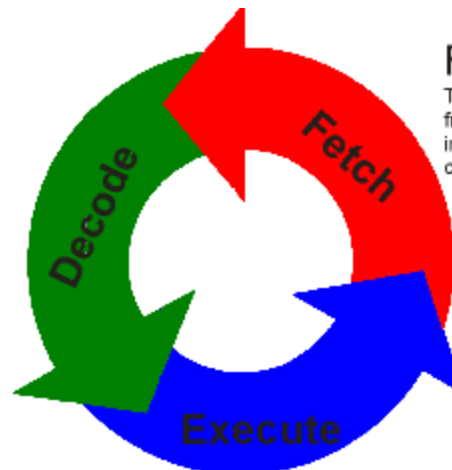
abstract, machine-independent, way to manipulate executable code at runtime

CPU's can **decode instructions** and **perform simple operations on register contents** at the rate of one or more operations per **clock cycle**

Instruction Cycle

- **Fetch:** get instruction from memory
 - **Increment PC**
- **Decode:** **translate** instruction into a form that the CPU can understand (**microcode**) and **fetch operands** if needed
- **Execute:** **perform** the **action** as specified in the microcode

Decode:
the bit pattern into
the instruction
register



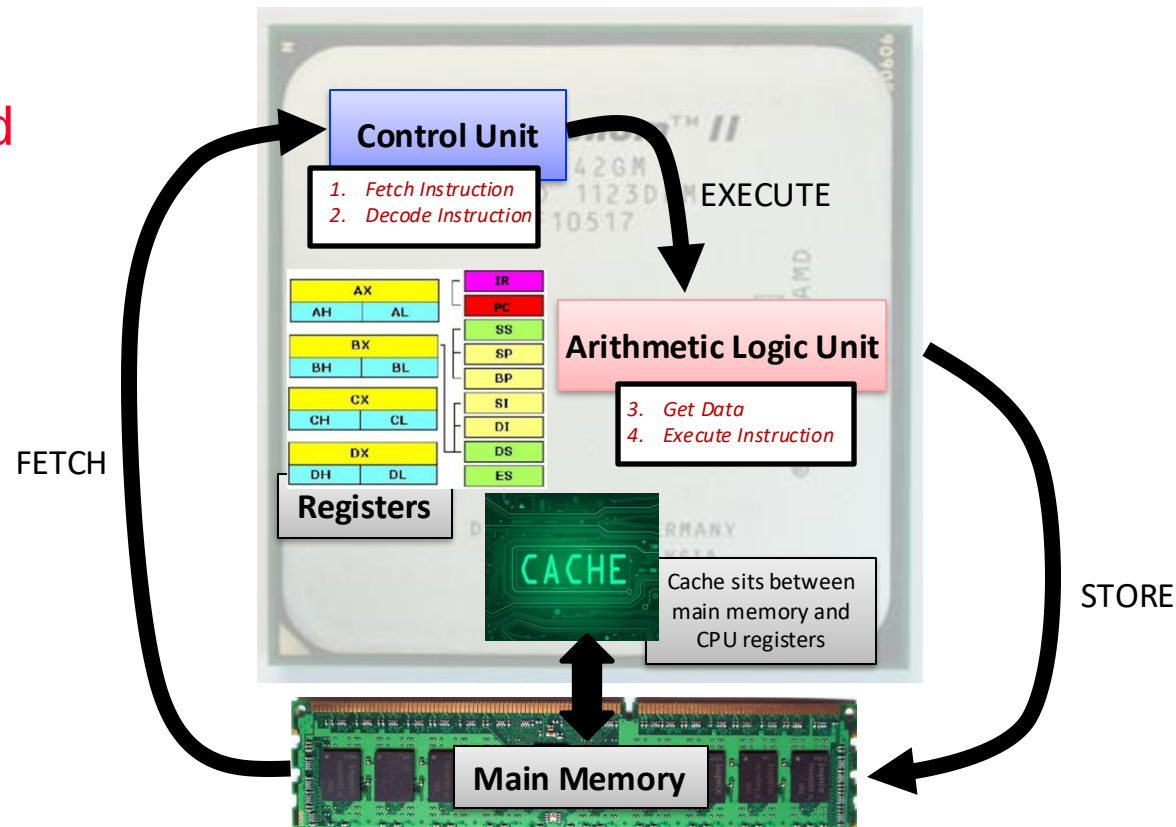
Fetch:
The next instruction
from memory and
increment the program
counter

Execute:
the action of the
instruction into the
instruction register

CPU Cycle Implementation

Fetch - Decode - Execute Cycle

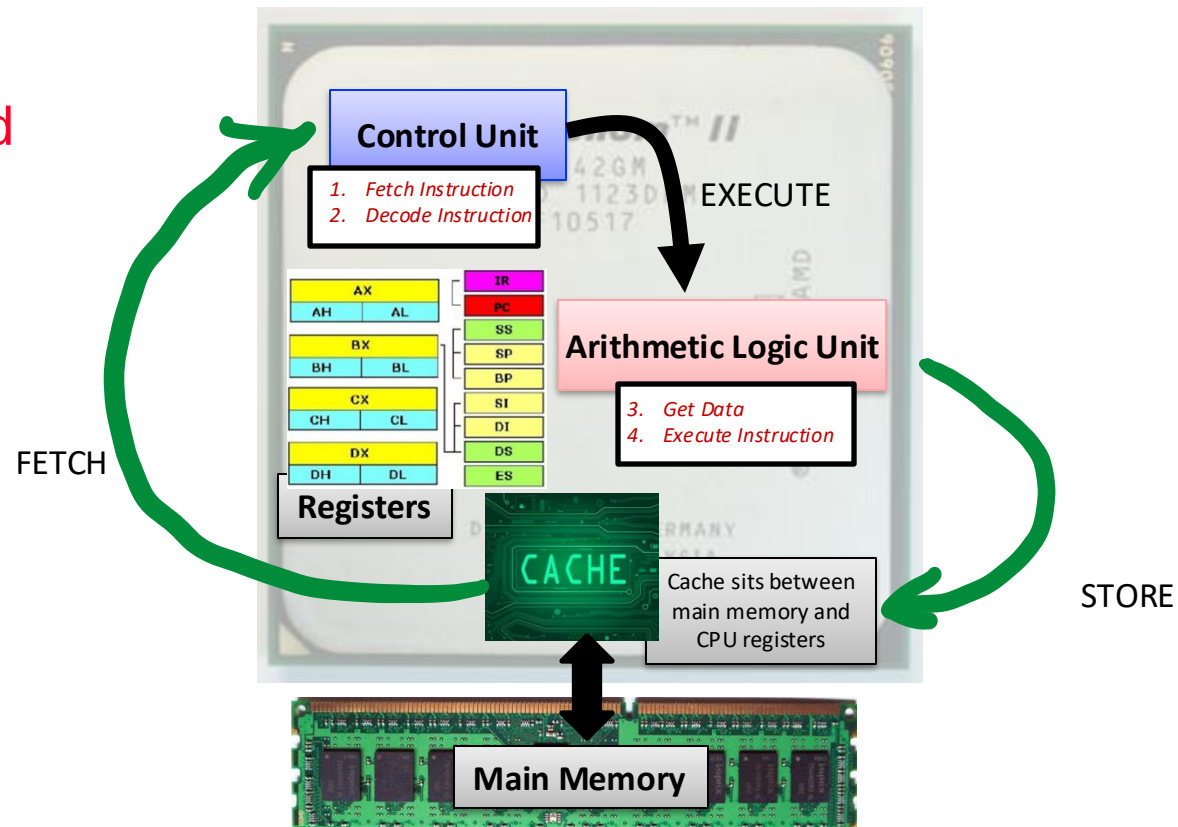
- **Program Counter:**
address of next
instruction to be fetched
- **Instruction Register:**
instruction currently
being decoded and/or
executed
- **CPU State:** a snapshot
of the values in **all**
different registers



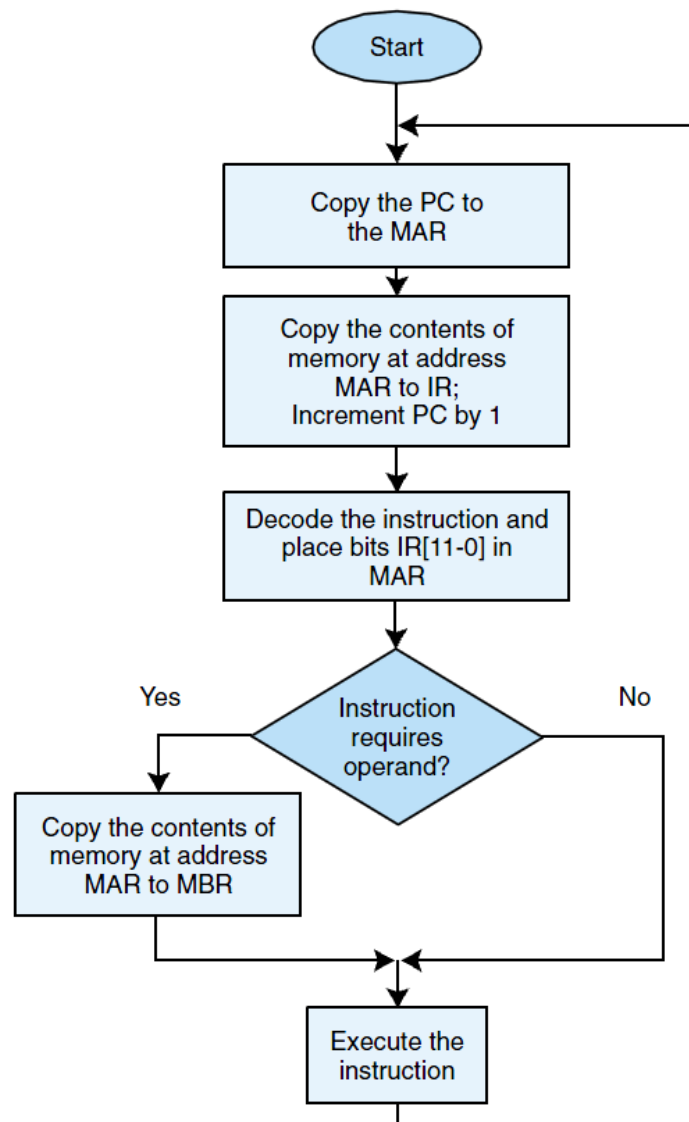
CPU Cycle Implementation

Fetch - Decode - Execute Cycle

- **Program Counter:** address of next instruction to be fetched
- **Instruction Register:** instruction currently being decoded and/or executed
- **CPU State:** a snapshot of the values in all different registers



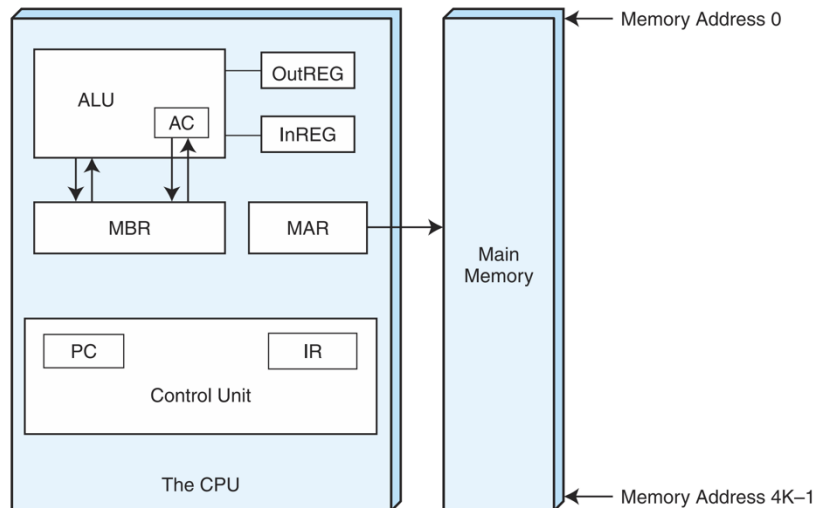
Microprocessor (Instruction Cycle)



EXAMPLE: 16-BIT CPU

(**MARIE**: Machine Architecture that is Really Intuitive and Easy)

MARIE

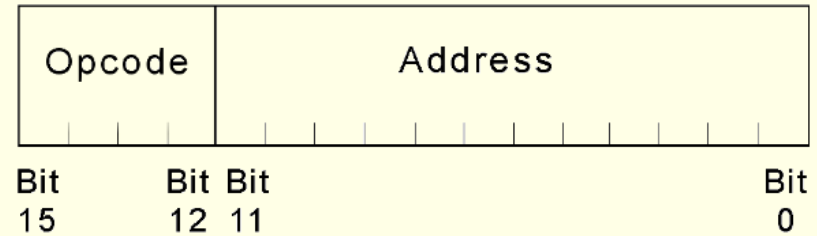


Binary, two's complement
 Stored program, fixed word length
 Word (but not byte) addressable
 4K words of main memory (this implies 12 bits per address)
 16-bit data (words have 16 bits)
 16-bit instructions, 4 for the opcode and 12 for the address
 A 16-bit accumulator (AC)
 A 16-bit instruction register (IR)
 A 16-bit memory buffer register (MBR)
 A 12-bit program counter (PC)
 A 12-bit memory address register (MAR)
 An 8-bit input register
 An 8-bit output register

- **AC:** The *accumulator*, which holds data values. This is a *general purpose register* and holds data that the CPU needs to process. Most computers today have multiple general purpose registers.
- **MAR:** The *memory address register*, which holds the memory address of the data being referenced.
- **MBR:** The *memory buffer register*, which holds either the data just read from memory or the data ready to be written to memory.
- **PC:** The *program counter*, which holds the address of the next instruction to be executed in the program.
- **IR:** The *instruction register*, which holds the next instruction to be executed.
- **InREG:** The *input register*, which holds data from the input device.
- **OutREG:** The *output register*, which holds data for the output device.

MARIE's Instruction Set

- This is the format of a MARIE instruction:



- The fundamental **MARIE** instructions are:

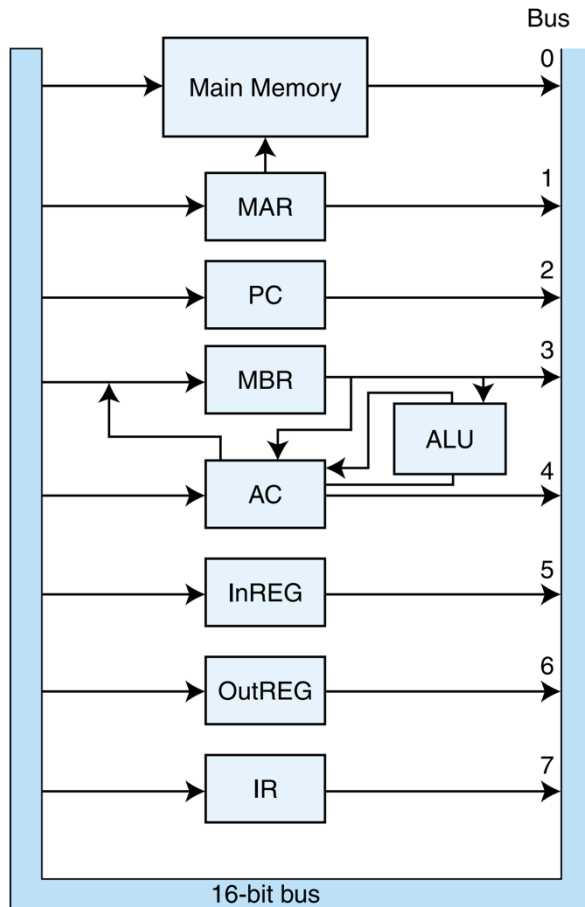
Instruction Number		Instruction	Meaning
Bin	Hex		
0001	1	Load X	Load the contents of address X into AC.
0010	2	Store X	Store the contents of AC at address X.
0011	3	Add X	Add the contents of address X to AC and store the result in AC.
0100	4	Subt X	Subtract the contents of address X from AC and store the result in AC.
0101	5	Input	Input a value from the keyboard into AC.
0110	6	Output	Output the value in AC to the display.
0111	7	Halt	Terminate the program.
1000	8	Skipcond	Skip the next instruction on condition.
1001	9	Jump X	Load the value of X into PC.

Register Transfer Language

Register Transfer Language or RTL shows how the CPU (Assembler) works using its main components:

- AC (*Accumulator*)
- PC (*Program Counter*)
- MAR (*Memory Address Register*)
- MBR (*Memory Buffer Register*)
- IR (*Instruction Register*)

Data Path and Inst. Implementation (Register Transfer Notation)



Load X: $MAR \leftarrow X$
 $MBR \leftarrow M[MAR], AC \leftarrow MBR$

Store X: $MAR \leftarrow X, MBR \leftarrow AC$
 $M[MAR] \leftarrow MBR$

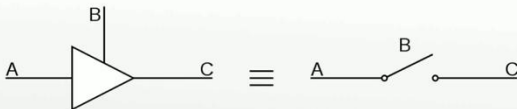
Add X: $MAR \leftarrow X$
 $MBR \leftarrow M[MAR]$
 $AC \leftarrow AC + MBR$

Subt X: $MAR \leftarrow X$
 $MBR \leftarrow M[MAR]$
 $AC \leftarrow AC - MBR$

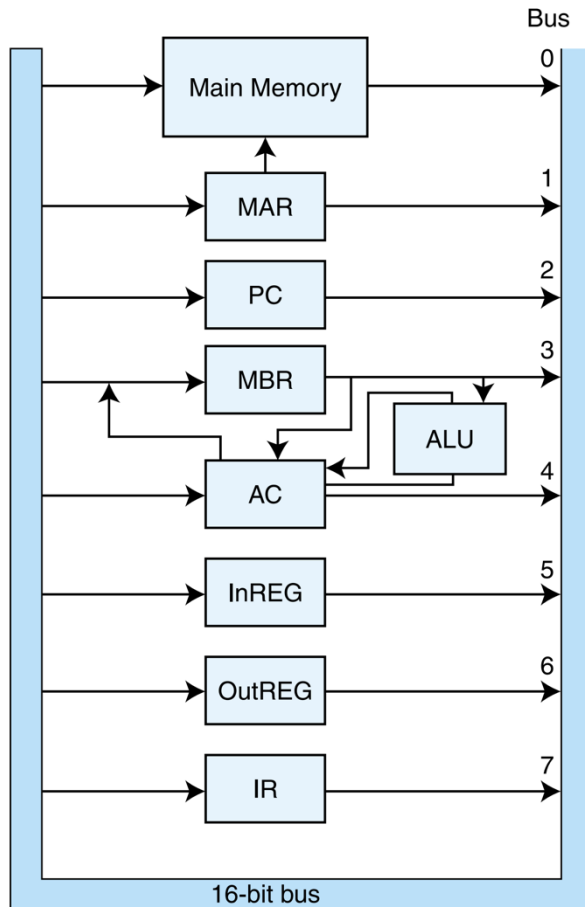
Input: $AC \leftarrow InREG$

Output: $OutREG \leftarrow AC$

Three-state logic



Data Path and Inst. Implementation



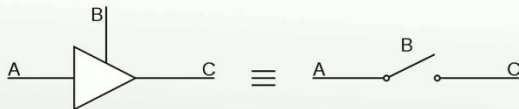
Jump X: $PC \leftarrow IR[11-0]$

Skipcond:

```

if IR[11-10] = 00 then           {if bits 10 and 11 in the IR are both 0}
    If AC < 0 then PC ← PC+1
else If IR[11-10] = 01 then      {if bit 11 = 0 and bit 10 = 1}
    If AC = 0 then PC ← PC + 1
else If IR[11-10] = 10 then      {if bit 11 = 1 and bit 10 = 0}
    If AC > 0 then PC ← PC + 1
    
```

Three-state logic



More on Decoding of Instructions

Hex Address	Instruction	Binary Contents of Memory Address	Hex Contents of Memory
100	Load 104	00010001000000100	1104
101	Add 105	00110001000000101	3105
102	Store 106	00100001000000110	2106
103	Halt	01110000000000000	7000
104	0023	00000000000100011	0023
105	FFE9	11111111111101001	FFE9
106	0000	00000000000000000	0000

Instruction Number		Instruction
Bin	Hex	
0001	1	Load X
0010	2	Store X
0011	3	Add X
0100	4	Subt X
0101	5	Input
0110	6	Output
0111	7	Halt
1000	8	Skipcond
1001	9	Jump X

Hex Address	Instruction	Binary Contents of Memory Address	Hex Contents of Memory
100	Load 104	0001000100000100	1104
101	Add 105	0011000100000101	3105
102	Store 106	0010000100000110	2106
103	Halt	0111000000000000	7000
104	0023	0000000000100011	0023
105	FFE9	111111111101001	FFE9
106	0000	0000000000000000	0000

a) Load 104

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		100	-----	-----	-----	-----
Fetch	$MAR \leftarrow PC$	100	-----	100	-----	-----
	$IR \leftarrow M[MAR]$	100	1104	100	-----	-----
	$PC \leftarrow PC + 1$	101	1104	100	-----	-----
Decode	$MAR \leftarrow IR[11-0]$	101	1104	104	-----	-----
	(Decode $IR[15-12]$)	101	1104	104	-----	-----
Get operand	$MBR \leftarrow M[MAR]$	101	1104	104	0023	-----
Execute	$AC \leftarrow MBR$	101	1104	104	0023	0023

Hex Address	Instruction	Binary Contents of Memory Address	Hex Contents of Memory
100	Load 104	00010001000000100	1104
101	Add 105	00110001000000101	3105
102	Store 106	00100001000000110	2106
103	Halt	01110000000000000	7000
104	0023	0000000000100011	0023
105	FFE9	1111111111101001	FFE9
106	0000	00000000000000000	0000

b) Add 105

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		101	1104	104	0023	0023
Fetch	$MAR \leftarrow PC$	101	1104	101	0023	0023
	$IR \leftarrow M[MAR]$	101	3105	101	0023	0023
	$PC \leftarrow PC + 1$	102	3105	101	0023	0023
Decode	$MAR \leftarrow IR[11-0]$	102	3105	105	0023	0023
	(Decode $IR[15-12]$)	102	3105	105	0023	0023
Get operand	$MBR \leftarrow M[MAR]$	102	3105	105	FFE9	0023
Execute	$AC \leftarrow AC + MBR$	102	3105	105	FFE9	000C

Hex Address	Instruction	Binary Contents of Memory Address	Hex Contents of Memory
100	Load 104	00010001000000100	1104
101	Add 105	00110001000000101	3105
102	Store 106	00100001000000110	2106
103	Halt	01110000000000000	7000
104	0023	0000000000100011	0023
105	FFE9	1111111111101001	FFE9
106	0000	00000000000000000	0000

c) Store 106

Step	RTN	PC	IR	MAR	MBR	AC
(initial values)		102	3105	105	FFE9	000C
Fetch	$MAR \leftarrow PC$	102	3105	102	FFE9	000C
	$IR \leftarrow M[MAR]$	102	2106	102	FFE9	000C
	$PC \leftarrow PC + 1$	103	2106	102	FFE9	000C
Decode	$MAR \leftarrow IR[11-0]$	103	2106	106	FFE9	000C
	(Decode $IR[15-12]$)	103	2106	106	FFE9	000C
Get operand	(not necessary)	103	2106	106	FFE9	000C
Execute	$MBR \leftarrow AC$	103	2106	106	000C	000C
	$M[MAR] \leftarrow MBR$	103	2106	106	000C	000C

Reading and Online Resources

- Get acquainted with the **MARIE simulator** (<https://marie.js.org/>) with a good resource that describes it here (<https://marie.js.org/book.pdf>)
- Please read and feel comfortable about:
 - Binary and Hexadecimal **number representation**
 - Number System Conversion
 - One's and Two's complement number representation
- Below are some resources:
 - https://www.tutorialspoint.com/basics_of_computers/basics_of_computers_number_system.htm
 - https://www.tutorialspoint.com/basics_of_computers/basics_of_computers_number_system_conversion.htm
 - https://en.wikipedia.org/wiki/Ones%27_complement
 - https://en.wikipedia.org/wiki/Two%27s_complement
- **Read Appendix A:** Instruction set Principles